

Cloud Computing CSC3074 Final Group Project: Scalable Application Architecture

This project requires your team to design, architect, and implement a fully functional, cloud-native application, demonstrating proficiency in core cloud services and architecture best practices.

1. Project Overview

Weight: 50% of total subject grade

Group Component: 30% (assessed as a group)

Individual Component: 20% (assessed individually)

Duration: 6 weeks (Weeks 11-17)

Team Size: 2-3 students per group

Submission Deadline: End of Week 16

2. Learning Outcome Assessed

This project assesses your ability to Create an application that incorporates the use of cloud virtual machine, cloud storage and/or cloud database (CLO3: C5, PLO3).

Project Goals

By completing this project, you will:

- i. Design and architect a cloud-native application using industry best practices
- ii. Implement a fully functional application deployed on cloud infrastructure
- iii. Integrate multiple cloud services (compute, storage, database)
- iv. Apply security measures, monitoring, and backup strategies
- v. Optimize for performance, scalability, and cost-effectiveness
- vi. Demonstrate effective collaboration and individual technical expertise
- vii. Document architectural decisions and technical implementations

3. Project Requirements: Mandatory Technical Components

Your application **MUST** integrate the following components, demonstrating proper configuration and inter-service connectivity:

3.1. Core Services (Required)

Component	Requirement Details
Cloud Virtual Machine(s)	At least one VM for application hosting or processing. Must be properly sized and configured for the workload, with configured networking and security groups.
Cloud Storage	Use Object Storage (e.g., S3/Blob Storage) for files, media, or backups, with proper access control and integration with the application.
Cloud Database	Use either a Relational (e.g., RDS, Azure SQL) OR NoSQL (e.g., DynamoDB, MongoDB Atlas) database, with a properly designed schema/structure and secure connection.

3.2. Scalability and Resilience (Required)

Your architecture must be designed for high availability and dynamic scaling.

- Load Balancing: Use an **Elastic Load Balancer (ELB)** or equivalent service to distribute incoming traffic across multiple running instances.
- Auto Scaling: Configure an **Auto Scaling Group** (or equivalent) to automatically increase/decrease the number of VM instances based on a defined metric (e.g., CPU utilization).
- Availability: Deploy instances across at least two separate Availability Zones for **fault tolerance**

3.3. Security & Monitoring

- Implement **Identity and Access Management (IAM)** roles/policies for secure access between services.
- Configure **Security Groups/Firewalls** to restrict traffic only to necessary ports and sources.
- Integrate **Cloud Monitoring** (e.g., CloudWatch) to track key metrics and set up basic alarms (e.g., for high CPU).

3.4. Testing & Operational Evaluation

- Explicitly trigger and document at least one cloud infrastructure event.
- Document functional connectivity validation demonstrating error-free data pipelines flowing securely among the Cloud VM, Object Storage API, and the Cloud Database.

4. Submission Deliverables

Your group is required to submit the following materials by the deadline:

4.1. Technical Report (Group Component)

Submit a comprehensive report covering the following sections:

- Architecture Design: Detailed diagrams and justification for all chosen services, regions, and instance types.
- Implementation & Evaluation Steps: A clear, step-by-step guide on deploying the cloud application, accompanied by log files, test cases, or monitoring dashboard screenshots verifying the successful integration, data persistence, and dynamic system failover.
- Security & Monitoring: Discussion of security measures implemented (IAM, security groups, encryption) and monitoring/scaling strategies (Auto Scaling configuration and logic).
- Challenges & Solutions: Documentation of technical challenges encountered and how they were resolved.
- Code Documentation: All code and documentation must be original or properly attributed. Use of AI tools must be disclosed in your report.

4.2. Presentation and Live Demonstration (Group Component)

A live, in-person group presentation showcasing the application's functionality, architecture, and its ability to scale and recover from failure.

4.3. Individual Contribution Statement (Individual Component)

A section within the final report where each member details their specific tasks, code contributions, and technical roles (e.g., "Deployed Auto Scaling," "Designed Database Schema"). This forms the basis for your individual grade.

Group Roles & Responsibilities

Suggested Role Distribution (2-Person Team)

Role 1: Cloud Architect & Security Lead

Primary Responsibilities:

- Design overall cloud architecture
- Set up VPC, subnets, and networking
- Configure security groups, IAM roles, and policies
- Implement WAF or security hardening
- Set up monitoring and alerts

Collaborative Responsibilities:

- Application development (shared)
- Testing and debugging (shared)

Role 2: Application Developer & Data Lead

Primary Responsibilities:

- Develop application code and logic
- Set up and configure database
- Implement storage integration
- Create backup policies
- Performance testing and optimization

Collaborative Responsibilities:

- Application development (shared)
- Testing and debugging (shared)

Suggested Role Distribution (3-Person Team)

Role 1: Cloud Infrastructure Engineer

Primary Responsibilities:

- Set up and configure cloud VMs
- Configure VPC, networking, and load balancing
- Implement auto-scaling
- Network security configuration
- Infrastructure as Code (bonus)

Role 2: Application Developer

Primary Responsibilities:

- Develop frontend and backend code
- Integrate cloud storage services
- Implement user authentication
- API development and integration
- Application-level error handling

Role 3: Security & Operations Engineer

Primary Responsibilities:

- Design and implement security architecture
- Configure IAM roles and RBAC
- Set up monitoring, logging, and alerts
- Create and test backup policies
- Implement database security
- Cost monitoring and optimization

Note: All team members should understand the entire system and contribute to documentation and testing.

Submission Requirements

Group Submissions

1. Architecture & Design Document (Week 11)
2. Project Report (Week 17)

Application Access Details to be included:

- Application URL
- Admin/test credentials
- Cloud console access (read-only IAM user)
- Git repository link

3. Presentation Slides (Week 17)

Submit before presentation session

Individual Submissions

1. Technical Implementation and Personal Reflection Report (Week 17)

Use naming convention: StudentID_FinalProject_Report.pdf

Frequently Asked Questions

Q: Can we use services beyond the mandatory components?

A: Yes! Additional services (e.g., CDN, caching, serverless functions) can earn bonus consideration in grading.

Q: What if team members contribute unequally?

A: Peer evaluations and individual git commits will be reviewed. Significant discrepancies may result in individual grade adjustments.

Q: Can we change team members after formation?

A: Only in exceptional circumstances with lecturer approval before Week 6.

Q: What if our application idea requires services not covered in class?

A: Schedule a meeting with the lecturer to discuss. You must still include all mandatory components.

Q: How much should we spend on cloud services?

A: Aim for \$0-10 using free tiers. Maximum \$20. Document all costs in your report.

Q: Can we use a PaaS platform like Heroku instead of VMs?

A: No. You must use IaaS components (VMs) to demonstrate CLO3. PaaS can be additional, not replacement.

Declaration

By submitting this project, all team members certify that:

- The work submitted is our own original work
- We have properly attributed any external resources used
- We understand the consequences of academic misconduct
- Individual components represent our personal contributions
- We have read and agree to the assessment requirements

Good luck with your project! Remember, this is your opportunity to demonstrate everything you've learned and build something you can showcase in your portfolio.

Appendix 1: Assessment Rubric

Group Component (30%)

Component	Excellent (80-100%)	Good (60-80%)	Satisfactory (40-60%)	Needs Improvement (<40%)
Architecture Document (8%)	Comprehensive, professional diagrams; excellent justifications; realistic budget	Clear diagrams; good justifications; reasonable budget	Basic diagrams; adequate justifications; budget present	Incomplete diagrams; weak justifications; no budget
Application Functionality (15%)	All features work flawlessly. All mandatory architecture layers are flawlessly configured and proven resilient under simulated failover/load testing with comprehensive evaluation data documented.	Most features work well; all core technical components (VM, Storage, DB) are integrated, but dynamic scaling/ELB shows minor config errors or delays under load.	Basic features work; application functions but missing one major technical component (e.g., no Auto-Scaling or no Object Storage configuration).	Many features broken; poor component integration; missing multiple mandatory infrastructure layers (e.g., no database connectivity and no scaling).

Group Presentation (7%)	Highly professional; excellent demo; deep technical knowledge	Clear and organized; good demo; solid understanding	Adequate presentation; demo works; basic understanding	Disorganized; demo issues; weak understanding
--------------------------------	---	---	--	---

Individual Component (20%)

Component	Excellent (80-100%)	Good (60-80%)	Satisfactory (40-60%)	Needs Improvement (<40%)
Personal contribution (5%)	Outlined distinct, quantifiable technical contributions verified by Git commits; demonstrated proactive technical initiative by independently designing/refactoring a core infrastructure layer or troubleshooting a bottleneck.	Provided clear, documented individual contributions to assigned tasks with solid Git history; successfully collaborated to integrate work with the team.	Contributions described with baseline Git commits; met basic assigned tasks but relied heavily on team members for integration.	Unclear or undocumented contributions; minimal or absent Git commit history; failed to complete assigned tasks.
Deep Dive on One Technical Area (5%)	Exceptional depth; Exceptional understanding	Good technical detail; Good understanding	Adequate detail; Adequate understanding	Lacks detail; Low to basic level of understanding
Individual Reflection (10%)	Highly critical and analytical reflection; clearly evaluates technical hurdles by explaining how specific cloud limitations (e.g., latency, cost, quotas) shaped design choices, offering viable architectural alternatives.	Descriptive and accurate reflection; successfully highlights personal challenges faced during deployment, lists the practical lessons learned from cloud setup, and documents	Basic reflection; provides a superficial overview of tasks completed and generic challenges faced without deep technical analysis.	Minimal reflection; mostly a summary of project requirements with no critical self-assessment or technical growth demonstrated.

		how problems were resolved.		
--	--	-----------------------------	--	--

Academic Integrity & Collaboration

Permitted Collaboration

- Team members working together on group components
- Using online resources, documentation, and tutorials
- Seeking help from lecturer during consultation hours
- Using AI tools for code suggestions (must be disclosed)

Not Permitted

- Copying code from other teams
- Submitting work done primarily by others as your own
- Plagiarizing documentation or reports
- Sharing complete solutions with other teams

Important: All code and documentation must be original or properly attributed. Use of AI tools must be disclosed in your report.

Appendix 2: Suggested Project Ideas

Option 1: Content Management & Sharing Platform

Users upload and share documents/images

- Uses: EC2/VMs, S3/Blob Storage, RDS/SQL Database
- Features: user authentication, file versioning, search

Option 2: E-Commerce or Inventory System

- Product catalog with shopping cart
- Uses: VMs, Object Storage for images, SQL/NoSQL database
- Features: order processing, inventory tracking, user accounts

Option 3: Data Analytics Dashboard

Collect, process, and visualize data

- Uses: VMs for processing, Storage for raw data, Database for processed data

- Features: data ingestion, real-time updates, visualization

Option 4: Booking/Reservation System

Appointment or resource booking application

- Uses: VMs for web server, Database for bookings, Storage for documents
- Features: calendar integration, notifications, reporting

Option 5: IoT Data Collection System

Collect and analyze sensor/device data

- Uses: VMs for API, Storage for raw data, NoSQL for time-series data
- Features: real-time monitoring, historical analysis, alerts

Option 6: Social Media or Forum Platform

User-generated content and interactions

- Uses: VMs, Storage for media, Database for posts/comments
- Features: user profiles, content moderation, notifications

Note: You may propose your own project idea, subject to lecturer approval by end of Week 9.